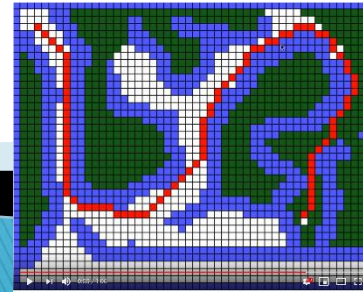


Professor Rong Qu
rong.qu@nottingham.ac.uk



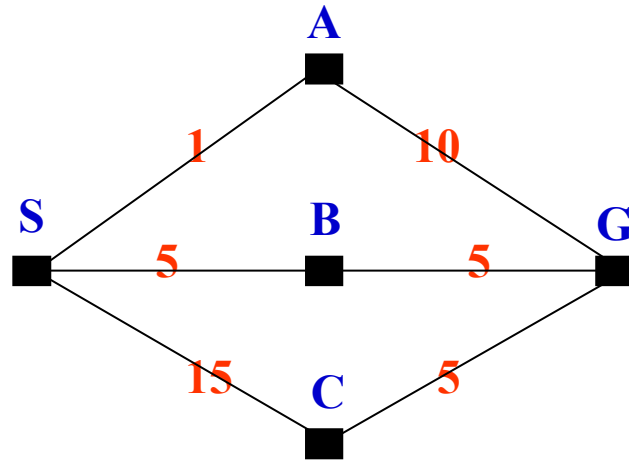
Search – General Search Algorithm

- ▶ Function **General-Search**(p, **QUEUING-FN**) returns a solution or failure
 - nodes = Make-Queue(Make-Node(Initial-State[p]))
 - Loop do
 - If nodes is empty then return failure
 - node = Remove-Front(nodes)
 - If Goal-Test[p] on State(node) succeeds then return node
 - nodes = **QUEUING-FN**(nodes, Expand(node, Operators[p]))
 - End
- ▶ End Function

Data structures: (**queue**)

Functions: **QUEUING-FN**

Uniform Cost Search (UCS)



- **BFS**: finds the path S-A-G (of cost 11)
- **UCS**: finds the cheaper solution (S-B-G). It finds S-A-G but doesn't expand it, as it is not the cheapest in the queue

```
▶ Function General-Search(p, QUEUING-FN) returns a solution or failure
  ◦ nodes = Make-Queue(Make-Node(Initial-State[p]))
  ◦ Loop do
    • If nodes is empty then return failure
    • node = Remove-Front(nodes)
    • If Goal-Test[p] on State(node) succeeds then return node
    • nodes = QUEUING-FN(nodes, Expand(node, Operators[p]))
  ◦ End
▶ End Function
```

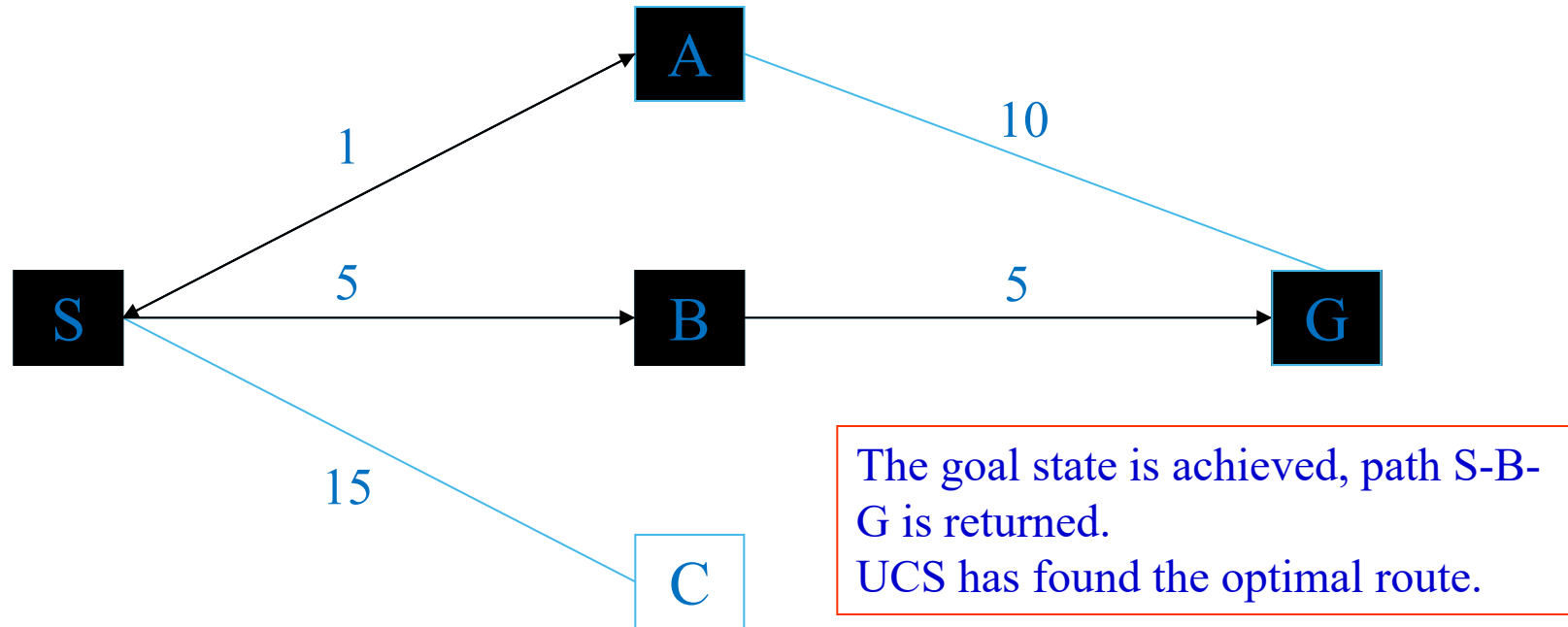
▶ UCS

- total cost of the path from the root to node n
- always remove the **smallest cost node** first

Function UCS(problem) returns a solution or failure

Return General-Search(problem, **ENQUEUE-BY-COST**)

Node B is removed from the queue, and the child A and child C are added. The queue is closed on again path. Don't add nodes with the same path. Since the queue is not empty, it is not the goal state. Node A is the root of the queue. Since the queue is not empty, we now proceed to goal state.



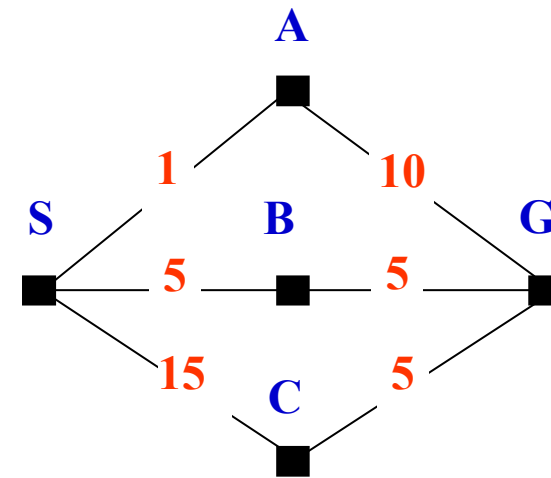
Press space to begin the search

Size of Queue: 0	Queue: Empty	
Nodes expanded: 3	FINISHED SEARCH	Current level: 2

UNIFORM COST SEARCH PATTERN

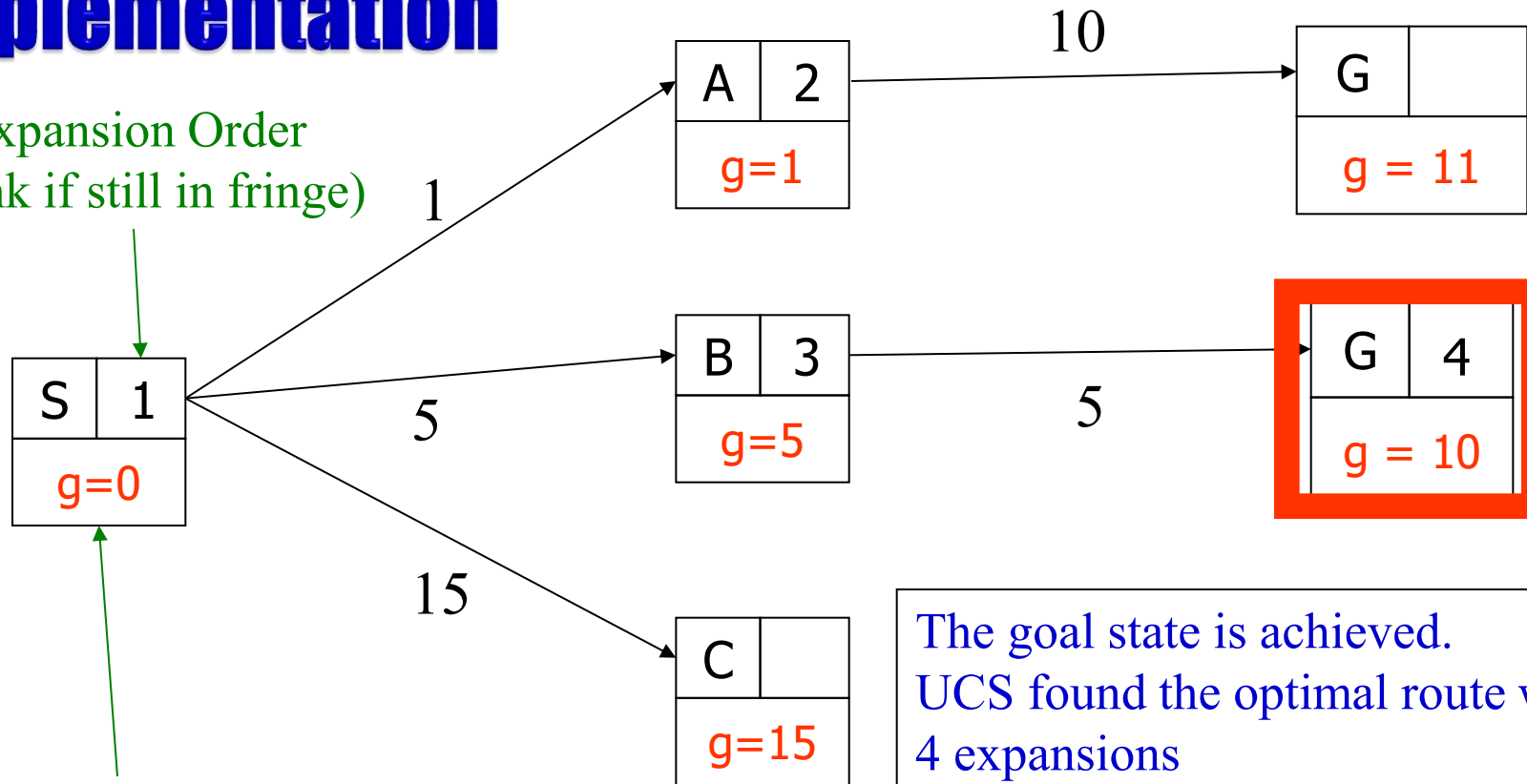
UCS – Implementation

Node Name	Expansion Number (order of expansion)
Any needed maths. i.e. values of g , h , and/or f	



UCS – Implementation

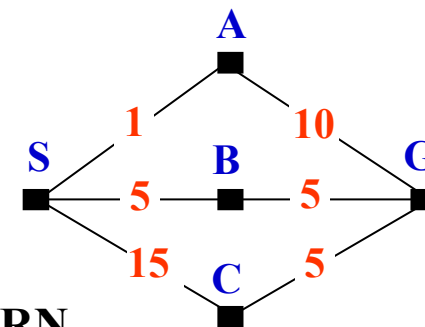
Expansion Order
(blank if still in fringe)



Maths: g

Note that G is split into two nodes corresponding to the two paths.

UNIFORM COST SEARCH PATTERN



UCS – Evaluation

- ▶ **Completeness**
 - Never miss a solution: from cheaper nodes to higher cost nodes: **yes**
- ▶ **Optimality**
 - Always explore the cheapest node (from the root) first: **yes**
- ▶ **Time complexity**
 - The worst case: b^d (although is often much better)
- ▶ **Space complexity**
 - Store: all the expanded nodes of the cheapest cost: b^d
- ▶ BFS: special case of UCS

Blind Searches – Summary

Evaluation	BFS & UCS	DFS
Time	b^d	b^m
Space	b^d	bm
Optimal?	Yes	No
Complete?	Yes	No

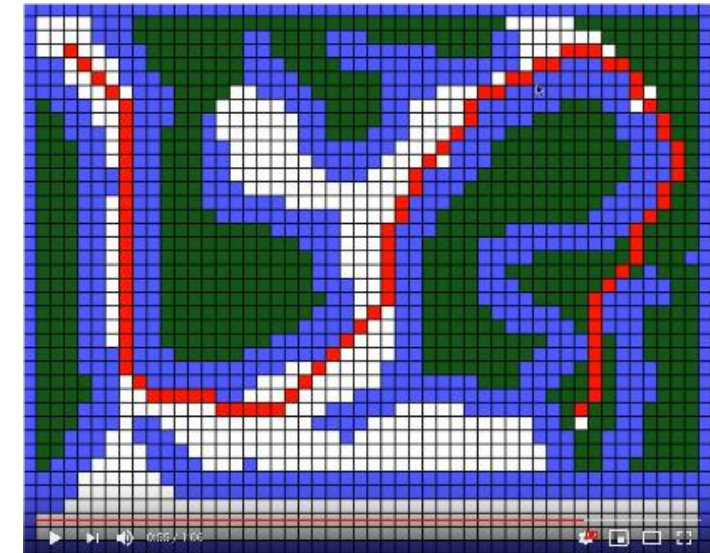
b: Average branching factor

d: Depth of solution

m: Maximum depth of the search tree

Note:

- BFS can be only applied to problems where costs are the same for each branch
- UCS can be applied to all problems as long as the costs never decrease along with the depth of the tree



Blind Searches – Past Exam Question

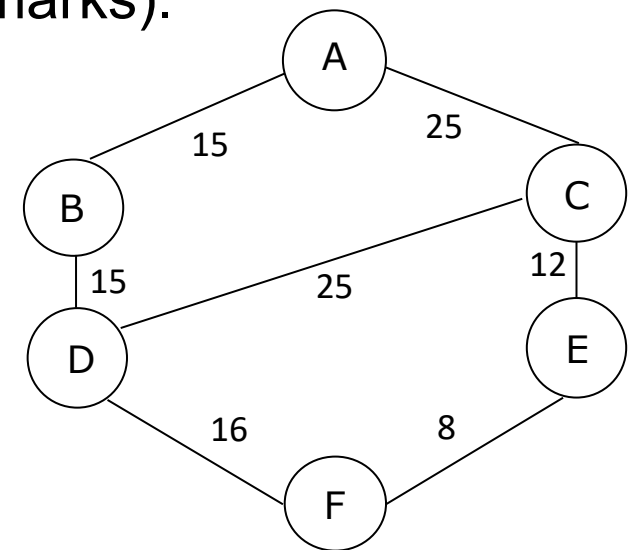
Use **Uniform Cost Search** to find the shortest path from A to F in the map below (not drawn to scale). You should not re-visit a node that you have just come from.

Show at each step what **fringe nodes** are in the queue (6 marks).

Show the list of nodes that are **expanded** (2 marks).

State the shortest route you take and its cost (2 marks).

Can Uniform Cost Search guarantee to find the optimal solution for this problem? Explain the reason. (3 marks)



Blind Searches – Summary

- ▶ The **General Search Algorithm**
 - Queue, Queuing Function
- ▶ **Four Criteria** for the evaluation of different search techniques / algorithms
 - Completeness, Optimality, Time complexity, Space complexity
 - Define and evaluate example problems with the criteria
 - **Compare** the different search techniques / algorithms

